Episode 3: Mistakes, Errors, Scenarios and Interview Answers

Common wrong ideas, real error messages and fixes, scenario questions, and solid interview answers with diagrams

Part A: Common mistakes and misunderstandings

Mistake	What really happens	Correct idea
"The Dockerfile is the image."	The Dockerfile is only text instructions.	docker build turns it into an image.
"docker build starts my app."	It builds an image. Nothing runs.	docker run starts a container from the image.
"CMD runs during the build."	CMD runs only when a container starts.	RUN is for build time, CMD for run time.
Editing the Dockerfile and expecting the running container to change	Running containers never change.	Rebuild the image, then replace the container.
Putting `COPY . .` before installing dependencies	Every code change reinstalls all libraries.	Copy requirements first, install, then copy code.
"EXPOSE publishes the port."	It is documentation only.	Publish with `-p HOST:CONTAINER`.
Flask or other apps listening on 127.0.0.1	The container is up but unreachable.	Listen on 0.0.0.0 inside the container.
Using many separate RUN lines for tiny steps	More layers and clutter.	Combine related commands, but keep readability.
Several CMD lines	Only the last CMD counts.	Use one CMD (and optionally one ENTRYPOINT).
Using `ADD` everywhere	It can fetch URLs and unpack archives unexpectedly.	Prefer COPY.
Building without a tag	The image is named latest or has no name.	Always tag: `-t name:1.0`.
Putting passwords in the Dockerfile	They stay visible in the image history.	Pass secrets at run time or use secret mounts (Episode 6 and 8).

Part B: Errors you will meet, and the fix

What you see	What it means	What to do
`failed to read dockerfile: open Dockerfile: no such file or directory`	Docker cannot find a file named Dockerfile in the context.	Run the command inside the project folder. Check the name: Dockerfile, capital D, no .txt. Or use -f.
`"docker build" requires exactly 1 argument`	You forgot the build context (the dot).	Add the context: `docker build -t name:1.0 .`
`failed to compute cache key ... "/requirements.txt": not found`	COPY cannot see that file in the context.	Check the file exists next to the Dockerfile, the spelling, and that .dockerignore does not exclude it.
`python:3.99-slim: not found`	That tag does not exist.	Look up valid tags on Docker Hub.
`pull access denied for pyton, repository does not exist`	Typo in the FROM image name.	Correct the name (python).
`ModuleNotFoundError: No module named 'flask'`	The library was not installed in the image.	Add flask to requirements.txt and keep the RUN pip install line. Rebuild.
Browser: empty response / connection reset, curl: (52) Empty reply from server	App listens on 127.0.0.1 or you mapped the wrong port.	Use host="0.0.0.0" in the app and check `-p HOST:5000`.
`address already in use`	The host port (often 5000 on a Mac) is taken.	Publish another port: `-p 8080:5000`.
`Conflict. The container name "/hello" is already in use`	An old container has that name.	`docker rm -f hello` then run again.
`exec format error`	The image was built for a different CPU type (for example built on an ARM Mac, run on an amd64 server).	Build for the target: `docker build --platform linux/amd64 ...`, or build multi-architecture images.
Container starts and exits at once	CMD missing, wrong, or the app crashed.	`docker logs NAME` and `docker ps -a` to see the exit code.
Single quotes in `CMD ['python','app.py']` then "not found"	Single quotes break the JSON array, so Docker treats it as a shell command.	Use double quotes in exec form.
Page still shows the old text after you edited the code	You did not rebuild, or you started the old container/image.	Rebuild, remove the old container, run the new image tag.

Debug order for builds: 1) read the **first red error line** and the instruction number it names. 2) confirm file names and paths relative to the context. 3) build again and watch which step fails. 4) run the image and use `docker logs`.

Part C: Scenario-based questions

Think first, then check the answer key in Part D.

Scenario 1: Old text after editing

You changed the message in app.py but the browser still shows the old text. **What did you forget, and what are the exact steps?**

Your answer: _____

Scenario 2: Slow rebuilds

Each small code change takes two minutes because all libraries are reinstalled. **What is wrong with the Dockerfile and how do you fix it?**

Your answer: _____

Scenario 3: Up but unreachable

The container shows Up in docker ps, the mapping is 8080:5000, yet the browser gets an empty response. **What do you check first?**

Your answer: _____

Scenario 4: Override the command

You want to print the Python version from the image without starting the app. **Which command, and what changes if the Dockerfile uses ENTRYPOINT ["python"]?**

Your answer: _____

Scenario 5: File not found

The build fails: COPY requirements.txt, "/requirements.txt": not found. **Give three likely causes.**

Your answer: _____

Scenario 6: Which instruction?

Name the instruction for each goal: (a) choose the Python version base, (b) install curl with apt-get, (c) set the folder commands run in, (d) define the default start command.

Your answer: _____

Scenario 7: Two versions at once

The team wants version 1.0 and 1.1 of the image available together. **How do you manage that?**

Your answer: _____

Scenario 8: Works on Mac, fails on server

An image built on an M-series Mac fails on a Linux amd64 server with exec format error. **Why, and how do you fix it?**

Your answer: _____

Scenario 9: Two CMD lines

A colleague wrote CMD ["python","app.py"] and later CMD ["python","worker.py"]. **Which one runs?**

Your answer: _____

Scenario 10: Explain to a manager

A manager asks: "What is the difference between the Dockerfile, the image and the container?" **Answer in three short sentences.**

Your answer: _____

Quick true or false

1. CMD runs while the image is being built. True / False
2. COPY and ADD behave exactly the same. True / False
3. EXPOSE 5000 makes the port reachable from the host. True / False
4. Changing a late line in the Dockerfile rebuilds only that layer and those after it. True / False
5. docker build produces an image, not a container. True / False
6. Only the last CMD in a Dockerfile takes effect. True / False

Part D: Answer key

- **Scenario 1:** You did not rebuild. Steps: edit, `docker build -t hello-flask:1.1 .`, `docker stop hello && docker rm hello`, then `docker run -d -p 8080:5000 --name hello hello-flask:1.1`.
- **Scenario 2:** `COPY . .` comes before the dependency install, so every code change invalidates the cache. Copy requirements.txt, run pip install, then copy the code.
- **Scenario 3:** Most likely the app listens on 127.0.0.1 inside the container. Change it to 0.0.0.0. Also check the app really listens on 5000 and read `docker logs`.
- **Scenario 4:** `docker run --rm hello-flask:1.0 python --version` works because the arguments replace CMD. With ENTRYPOINT ["python"], run `docker run --rm hello-flask:1.0 --version`, since the arguments are appended to python.
- **Scenario 5:** (1) Wrong folder: you ran the command from a different directory. (2) The file name is misspelled or missing. (3) .dockerignore excludes it, or the path is outside the build context.
- **Scenario 6:** (a) FROM (b) RUN apt-get update && apt-get install -y curl (c) WORKDIR (d) CMD (or ENTRYPOINT with CMD).
- **Scenario 7:** Use tags: build `hello-flask:1.0` and `hello-flask:1.1`. Add alias tags with `docker tag`. Run each as a separate container with different host ports.
- **Scenario 8:** The image was built for ARM, the server is amd64. Build for the target with `--platform linux/amd64` or publish a multi-architecture image.
- **Scenario 9:** Only the last one: worker.py runs. Use one CMD per Dockerfile, or separate images or an override for multiple programs.
- **Scenario 10:** The Dockerfile is the written recipe. The image is the sealed kit built from it. The container is a running copy of the image.

True or false answers

1. **False** (run time) | 2. **False** (ADD can fetch URLs and extract archives) | 3. **False** (documentation only) | 4. **True** | 5. **True** | 6. **True**

Part E: Interview Corner - solid answers with diagrams

Use these as a base and say them in your own words. Examples from your own lab make your answers stronger.

> **Answer formula: 1. Define** in one sentence. **2. Explain how** it works. **3. Give an example** instruction or command. **4. Mention a trade-off or pitfall.**

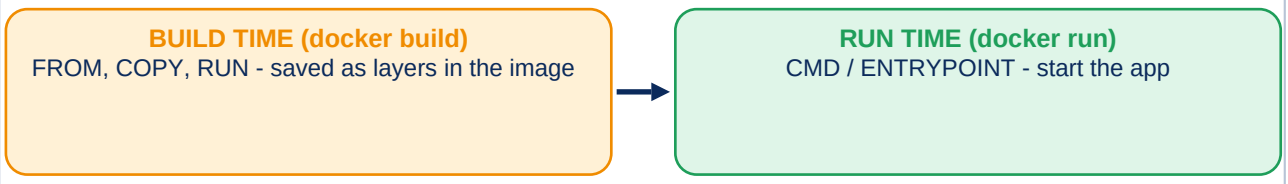
Interview Q1. What is a Dockerfile, and what does `docker build` do?

Say this first (30 sec)	A **Dockerfile** is a text file of instructions that defines how to build an image. `docker build` sends the **build context** to the Docker builder, executes the instructions in order, and produces an image, tagged with the name you give.
Add depth	Dockerfiles make builds **repeatable and reviewable**: they live in Git next to the code and CI can build the same image on every commit. Building creates an image only; a container exists after `docker run`.
Example to adapt	"Our CI runs docker build on each commit and tags the image with the Git commit ID."
Likely follow-up	"Where does the build happen?" In the Docker daemon (via BuildKit in current versions), which is why the context is sent to it.

Interview Q2. Walk me through a simple Python Dockerfile.

Say this first (30 sec)	**FROM** picks the base image, **WORKDIR** sets the working folder, **COPY requirements.txt** then **RUN pip install** installs dependencies, **COPY . .** adds the code, **EXPOSE** documents the port, and **CMD** defines how to start the app.
Add depth	I copy requirements first to use the layer cache. I use a pinned, slim base image for smaller size and reproducibility. The app binds to 0.0.0.0 so the published port works. Later improvements: .dockerignore, multi-stage builds and a non-root user.
Example to adapt	"For a Flask app I used python:3.11-slim, installed flask with pip, and started it with CMD in exec form."
Likely follow-up	"Why slim and not alpine?" Slim is Debian-based, smaller than full, and avoids musl-related compatibility issues some Python packages hit on Alpine.

Interview Q3. Explain RUN vs CMD vs ENTRYPOINT.

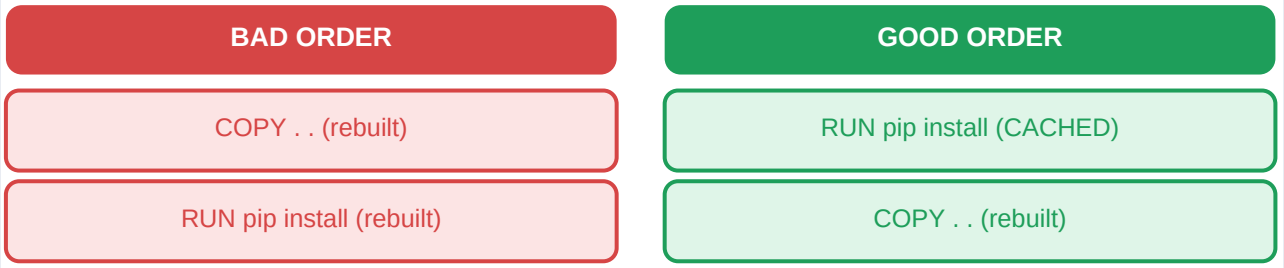


Say this first (30 sec)	**RUN** executes at **build time** and stores the result in a layer. **CMD** sets the **default command** at container start and is easy to override. **ENTRYPOINT** sets the **fixed executable**; arguments from docker run are appended to it.
Add depth	A common pattern is ENTRYPOINT for the program and CMD for its default arguments, so users can change the arguments but not the program. Only the last CMD and the last ENTRYPOINT in a Dockerfile count.
Example to adapt	"With ENTRYPOINT [\"python\"] and CMD [\"app.py\"], docker run IMG other.py runs python other.py."
Likely follow-up	"How do you override an ENTRYPOINT?" With the `--entrypoint` flag on docker run.

Interview Q4. What is the difference between exec form and shell form, and why does it matter?

Say this first (30 sec)	**Exec form** is a JSON array, `CMD ["python","app.py"]`. The process runs directly as the container's main process (PID 1). **Shell form** is `CMD python app.py`, which runs under `/bin/sh -c`.
Add depth	With exec form the app receives stop signals (SIGTERM) directly, so `docker stop` can shut it down gracefully. In shell form the shell may sit in front and not pass signals on, causing slow stops and killed processes. Shell form does allow variable expansion and shell features.
Example to adapt	"I switched to exec form after docker stop always took 10 seconds."
Likely follow-up	"Why must you use double quotes?" The array must be valid JSON. With single quotes Docker silently treats it as shell form.

Interview Q5. How do image layers and the build cache work? How do you optimise them?



Say this first (30 sec)	Instructions such as RUN, COPY and ADD create **read-only layers**. Docker caches each layer and reuses it if the instruction and its input files are unchanged. When a layer changes, **every layer after it is rebuilt**.
Add depth	Optimise by ordering: stable things first (base image, dependency files, install), frequently changing things last (source code). Combine related commands, clean package caches in the same RUN, and use a .dockerignore file so irrelevant file changes do not break the cache. Multi-stage builds (Episode 7) shrink the final image.
Example to adapt	"Moving COPY requirements.txt and pip install above COPY . . cut our rebuild from minutes to seconds."
Likely follow-up	"Does an unchanged Dockerfile always hit the cache?" Not if the copied files changed, or if the base image tag moved and you pulled a new one.

Interview Q6. COPY vs ADD: which do you use and why?

Say this first (30 sec)	I use **COPY**. It copies files and folders from the build context into the image. **ADD** does that and can also fetch remote URLs and automatically extract local tar archives.
Add depth	That extra behaviour is less predictable and URLs fetched by ADD are harder to cache and verify. Use ADD only when you need the tar auto-extraction. For remote files it is usually clearer to use RUN with curl and checksum verification.
Example to adapt	"We replaced ADD with COPY after an archive extracted into the wrong folder."
Likely follow-up	"Can COPY use files outside the context?" No. Only files inside the build context are available.

Interview Q7. What does EXPOSE do? How is it different from -p?

Say this first (30 sec)	**EXPOSE** documents that the container listens on a port. It does not open or publish anything. `-p HOST:CONTAINER` at run time actually publishes the port to the host.
Add depth	EXPOSE helps readers and tools. `docker run -P` (capital) publishes all exposed ports to random host ports. Containers on the same Docker network can reach each other's ports without publishing (Episode 4).
Example to adapt	"EXPOSE 5000 in the Dockerfile and -p 8080:5000 at run time."
Likely follow-up	"Is the app reachable without EXPOSE?" Yes, if you publish the port with -p; EXPOSE is not required for that.

Interview Q8. What is the build context and why does the dot matter?	
Say this first (30 sec)	The **build context** is the set of files Docker can access during the build. In `docker build -t app:1.0 .`, the final dot says the current folder is the context. COPY can only use files inside it.
Add depth	The whole context is sent to the builder, so a huge folder slows builds. Use `.dockerignore` to exclude things such as .git, node_modules, virtual environments and local secrets (Episode 7). Use `-f` to point at a Dockerfile in a different place.
Example to adapt	"After adding a .dockerignore the context went from 800 MB to 5 MB."
Likely follow-up	"Does the Dockerfile have to be inside the context?" Usually yes, but -f can reference another location.

Interview Q9. How do you tag and version images?	
Say this first (30 sec)	Use `-t name:tag` at build time, for example `hello-flask:1.0`. Add more tags with `docker tag`. Tags are labels pointing to an image, so one image can have several.
Add depth	Avoid depending on `latest` in production: it is just the default tag, not a guarantee of the newest build. A practical scheme: a version tag (1.4.2) plus the Git commit ID, and optionally a moving alias like stable. For strict reproducibility, deploy by image **digest**.
Example to adapt	"CI tags every build with the commit ID and promotes a tested tag to release."
Likely follow-up	"Can tags change?" Yes, a tag can be moved to another image; a digest cannot.

Interview Q10. SCENARIO: Builds are slow because dependencies reinstall every time. Fix it.	
BAD ORDER COPY . . (rebuilt) RUN pip install (rebuilt)	**GOOD ORDER** RUN pip install (CACHED) COPY . . (rebuilt)
Say this first (30 sec)	The dependency install layer is being invalidated by an earlier `COPY . .`. I copy only the dependency file first, run the install, and copy the source afterwards.
Add depth	Also add a .dockerignore, pin dependency versions so cache results are predictable, and consider BuildKit cache mounts for package downloads. In CI, reuse cache between runs.
Example to adapt	"We reordered the Dockerfile and rebuilds dropped from about two minutes to a few seconds." (Use your own numbers only if measured.)
Likely follow-up	"Why did the cache still miss?" The requirements file changed, the base image updated, or a previous layer changed.

Interview Q11. SCENARIO: The image runs on my Mac but fails on the Linux server with exec format error.	
Say this first (30 sec)	The image architecture does not match the server CPU. My Mac is ARM (arm64) and the server is amd64, so the binaries cannot run.
Add depth	Build for the target with `--platform linux/amd64`, or build a multi-architecture image with `docker buildx` so each machine pulls its own variant. Building in CI on the target architecture also avoids the problem.
Example to adapt	"We publish multi-architecture images so Apple silicon laptops and amd64 servers both work."
Likely follow-up	"How do you check an image's architecture?" `docker image inspect` shows Architecture and Os.

Interview Q12. SCENARIO: The container is Up but the app is unreachable from the browser.	
Say this first (30 sec)	I check from the outside in: `docker ps` for the mapping (HOST:CONTAINER) and that the container port matches the app's port; `docker logs` to see which address and port the app bound to. The classic cause is the app listening on **127.0.0.1** instead of **0.0.0.0**.
Add depth	Inside a container, 127.0.0.1 refers only to that container, so traffic arriving via the published port cannot reach it. Fix the app config (for Flask, host="0.0.0.0"). Then check firewalls or a host port clash.
Example to adapt	"I changed app.run() to host 0.0.0.0, rebuilt, and the page loaded."
Likely follow-up	"Is 0.0.0.0 safe?" Inside a container, yes, because exposure is still controlled by -p and the host firewall.

Interview Q13. Why not put secrets such as passwords in the Dockerfile?

Say this first (30 sec)	Anything written with ENV, ARG or RUN is stored in the image layers and history. Anyone who can pull the image can read it with `docker history` or inspect the layers, even if you delete it in a later step.
Add depth	Pass secrets at run time (environment variables from a secret store, mounted files) or use BuildKit secret mounts when a build needs a credential. Never commit secrets to the Dockerfile or the repo. (Episodes 6 and 8.)
Example to adapt	"We use BuildKit secret mounts so the token is never written into a layer."
Likely follow-up	"Does deleting the file in a later layer remove it?" No. The earlier layer still contains it.

Rapid-fire round (one-line answers)

Question	One-line answer
What does docker build produce?	An image.
Which instruction chooses the base image?	FROM
RUN or CMD runs at build time?	RUN
Does EXPOSE publish a port?	No, use -p.
Which CMD counts if there are two?	The last one.
Best order for dependencies and code?	Dependencies first, code last.
COPY or ADD by default?	COPY
Which form lets the app get stop signals?	Exec form (JSON array).
What does -t do?	Names and tags the image.
Edited the Dockerfile; what next?	Rebuild the image, start a new container.

Practice tip: Write the Dockerfile from memory on paper, then explain each line out loud and say what changes if you swap two lines. That is exactly how interviewers test understanding.